

Kickoff Protocol Contracts

Kickoff.fun

HALBORN

Kickoff Protocol Contracts - Kickoff.fun

Prepared by:  HALBORN

Last Updated 01/20/2026

Date of Engagement: January 6th, 2026 - January 9th, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
19	0	0	10	9	0

TABLE OF CONTENTS

1. Introduction	
2. Assessment summary	
3. Test approach and methodology	
4. Risk methodology	
5. Scope	
6. Assessment summary & findings overview	
7. Findings & Tech Details	
7.1 Nfts with zero voting power can permanently dos vote casting	
7.2 Deactivated veaero nfts can permanently dos voting batches and trap funds	
7.3 Emergency withdrawals break core accounting invariants and can permanently deadlock the pool	
7.4 Kickoff pool can accept non-votable nfts due to epoch-start mismatch between kickoff and aerodrome	
7.5 Pool creation can be front-run to steal privileged roles and block the intended launch	
7.6 Liquidity finalization can be dosed due to manipulated pool ratios	
7.7 Pools can finalize with zero participants and permanently lock project tokens	
7.8 Unintended liquidity creation can permanently consume project tokens due to external weth transfers	
7.9 Reward conversion may execute with unbounded slippage	
7.10 Kickoff finalization can occur before aerodrome epoch completion, causing reward loss	
7.11 Gauge status change between vote batches can permanently brick voting and trap nfts	
7.12 Claim rounding leaves irrecoverable project token dust in the pool	

7.13 Swap and liquidity operations lack meaningful deadline constraints

7.14 Rescue mechanism fails for tokens that do not return a boolean on successful transfers

7.15 Batch voting can span multiple aerodrome epochs, leading to inconsistent reward attribution

7.16 Locking veaero nfts allows kickoff to claim historical voting rewards

7.17 Lplocker fee distribution can be permanently dosed by restrictive project token behavior

7.18 Leftover liquidity allocation tokens can become permanently stuck after adding liquidity

7.19 Early emergency exit can permanently lock allocated project tokens and allow new locks into an unusable pool

1. INTRODUCTION

Kickoff.fun engaged Halborn to conduct a security assessment on their smart contracts beginning on January 6th, 2026 and ending on January 9th, 2026. The security assessment was scoped to the smart contracts provided in the **kickoff-contracts** GitHub repository, provided to the Halborn team. Commit hash and further details can be found in the Scope section of this report.

The reviewed contracts implement a liquidity bootstrapping launchpad built on Aerodrome's veAERO voting system. veAERO holders temporarily lock NFTs to direct gauge votes, after which rewards are claimed, swapped, and used to create permanently locked liquidity, while participants receive project tokens proportionally.

2. ASSESSMENT SUMMARY

Halborn was provided with 4 days for this engagement and assigned a full-time security engineer to assess the security of the smart contracts in scope. The assigned engineer possesses deep expertise in blockchain and smart contract security, including hands-on experience with multiple blockchain protocols.

The objective of this assessment is to:

- Identify potential security issues within the **Kickoff.fun** protocol smart contracts.
- Ensure that smart contract of **Kickoff.fun** protocol functions operate as intended.

In summary, **Halborn** identified several areas for improvement to reduce the likelihood and impact of security risks, which were mostly addressed by the **Kickoff.fun** team. The main recommendations were:

- **Handle batch voting failures at the per-NFT level to avoid single-point batch failures.**
- **Require pool activation to be aligned with Aerodrome's epoch start.**
- **Design liquidity finalization to safely handle pre-existing pool state.**
- **Provide a defined path for zero-participant scenario to avoid project tokens becoming stranded in the pool.**
- **Implement meaningful slippage protection when swapping reward tokens.**

3. TEST APPROACH AND METHODOLOGY

Halborn performed a combination of manual review of the code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of the smart contract assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of smart contracts and can quickly identify items that do not follow security best practices.

The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture and purpose of the **Kickoff.fun** protocol.
- Manual code review and walkthrough of the **Kickoff.fun** in-scope contracts.
- Manual assessment of critical **Solidity** variables and functions to identify potential vulnerability classes.
- Manual testing using custom scripts.
- Static Analysis of security for scoped contract, and imported functions. (**Slither**).
- Local deployment and testing with (**Foundry** , **Remix IDE**).

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

REPOSITORY ^

(a) Repository: [kickoff-contracts](#)

(b) Assessed Commit ID: [f3b6390](#)

(c) Items in scope:

- [script/Deploy.s.sol](#)
- [src/interfaces/IERC20.sol](#)
- [src/interfaces/IERC721Receiver.sol](#)
- [src/interfaces/IPool.sol](#)
- [src/interfaces/IRouter.sol](#)
- [src/interfaces/IVoter.sol](#)
- [src/interfaces/IVotingEscrow.sol](#)
- [src/interfaces/IVotingReward.sol](#)
- [src/interfaces/IWETH.sol](#)
- [src/libraries/EpochLib.sol](#)
- [src/LPLocker.sol](#)
- [src/KickoffFactory.sol](#)
- [src/KickoffVoteSalePool.sol](#)

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID: ^

- [98e47ac](#)

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL 0	HIGH 0	MEDIUM 10	LOW 9
INFORMATIONAL 0			

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
NFTS WITH ZERO VOTING POWER CAN PERMANENTLY DOS VOTE CASTING	MEDIUM	SOLVED - 01/13/2026
DEACTIVATED VEAERO NFTS CAN PERMANENTLY DOS VOTING BATCHES AND TRAP FUNDS	MEDIUM	SOLVED - 01/13/2026
EMERGENCY WITHDRAWALS BREAK CORE ACCOUNTING INVARIANTS AND CAN PERMANENTLY DEADLOCK THE POOL	MEDIUM	SOLVED - 01/13/2026
KICKOFF POOL CAN ACCEPT NON-VOTABLE NFTS DUE TO EPOCH-START MISMATCH BETWEEN KICKOFF AND AERODROME	MEDIUM	SOLVED - 01/13/2026
POOL CREATION CAN BE FRONT-RUN TO STEAL PRIVILEGED ROLES AND BLOCK THE INTENDED LAUNCH	MEDIUM	SOLVED - 01/13/2026
LIQUIDITY FINALIZATION CAN BE DOSED DUE TO MANIPULATED POOL RATIOS	MEDIUM	SOLVED - 01/13/2026
POOLS CAN FINALIZE WITH ZERO PARTICIPANTS AND PERMANENTLY LOCK PROJECT TOKENS	MEDIUM	SOLVED - 01/13/2026
UNINTENDED LIQUIDITY CREATION CAN PERMANENTLY CONSUME PROJECT TOKENS DUE TO EXTERNAL WETH TRANSFERS	MEDIUM	SOLVED - 01/13/2026
REWARD CONVERSION MAY EXECUTE WITH UNBOUNDED SLIPPAGE	MEDIUM	SOLVED - 01/13/2026
KICKOFF FINALIZATION CAN OCCUR BEFORE AERODROME EPOCH COMPLETION, CAUSING REWARD LOSS	MEDIUM	SOLVED - 01/13/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDATION DATE
GAUGE STATUS CHANGE BETWEEN VOTE BATCHES CAN PERMANENTLY BRICK VOTING AND TRAP NFTS	LOW	SOLVED - 01/13/2026
CLAIM ROUNDING LEAVES IRRECOVERABLE PROJECT TOKEN DUST IN THE POOL	LOW	SOLVED - 01/18/2026
SWAP AND LIQUIDITY OPERATIONS LACK MEANINGFUL DEADLINE CONSTRAINTS	LOW	SOLVED - 01/13/2026
RESCUE MECHANISM FAILS FOR TOKENS THAT DO NOT RETURN A BOOLEAN ON SUCCESSFUL TRANSFERS	LOW	SOLVED - 01/13/2026
BATCH VOTING CAN SPAN MULTIPLE AERODROME EPOCHS, LEADING TO INCONSISTENT REWARD ATTRIBUTION	LOW	SOLVED - 01/16/2026
LOCKING VEAERO NFTS ALLOWS KICKOFF TO CLAIM HISTORICAL VOTING REWARDS	LOW	RISK ACCEPTED - 01/19/2026
LPLOCKER FEE DISTRIBUTION CAN BE PERMANENTLY DOSED BY RESTRICTIVE PROJECT TOKEN BEHAVIOR	LOW	SOLVED - 01/13/2026
LEFTOVER LIQUIDITY ALLOCATION TOKENS CAN BECOME PERMANENTLY STUCK AFTER ADDING LIQUIDITY	LOW	SOLVED - 01/18/2026
EARLY EMERGENCY EXIT CAN PERMANENTLY LOCK ALLOCATED PROJECT TOKENS AND ALLOW NEW LOCKS INTO AN UNUSABLE POOL	LOW	SOLVED - 01/13/2026

7. FINDINGS & TECH DETAILS

7.1 NFTS WITH ZERO VOTING POWER CAN PERMANENTLY DOS VOTE CASTING

// MEDIUM

Description

Pool creation allows `minVotingPower` to be set to zero, and `lockVeAERO()` only checks `votingPowerAmount < minVotingPower`. If `minVotingPower` is zero, an NFT with zero voting power can be locked successfully.

```
106  /// @param minVotingPower Minimum voting power required to lock veAERO NFT (0 = no minimum)
107  /// @return pool The address of the created pool
108  function createPool(
109      address projectToken,
110      address projectOwner,
111      uint256 totalAllocation,
112      uint256 minVotingPower
113  ) external returns (address pool) {...}
```

```
294  function lockVeAERO(uint256 tokenId) external nonReentrant inState(PoolState.Active) {
295      //...
296
297      // Get voting power
298      uint256 votingPowerAmount = votingEscrow.balanceOfNFT(tokenId);
299
300      // Check minimum voting power requirement
301      if (votingPowerAmount < minVotingPower) {
302          revert VotingPowerTooLow();
303      }
304
305      //...
306  }
```

When voting is later executed, `castVotes()` or `castVotesBatch()` will attempt to vote with all locked NFTs. Aerodrome's voter contract reverts when voting with a zero-power NFT with `Voter.ZeroBalance()` error, causing the voting transaction to fail.

```
3004  // @ Voter._vote()
3005  // @ https://vscode.blockscan.com/8453/0x16613524e02ad97edfef371bc883f2f5d6c480a5
3006  function _vote(uint256 _tokenId, uint256 _weight, address[] memory _poolVote, uint256[] memory
3007      _reset(_tokenId);
3008      uint256 _poolCnt = _poolVote.length;
3009      uint256 _totalVoteWeight = 0;
3010      uint256 _totalWeight = 0;
3011      uint256 _usedWeight = 0;
3012
3013      for (uint256 i = 0; i < _poolCnt; i++) {
3014          _totalVoteWeight += _weights[i];
3015      }
3016
3017      for (uint256 i = 0; i < _poolCnt; i++) {
3018          address _pool = _poolVote[i];
3019          address _gauge = gauges[_pool];
3020          if (_gauge == address(0)) revert GaugeDoesNotExist(_pool);
3021      }
```

```

3022         if (!isAlive[_gauge]) revert GaugeNotAlive(_gauge);
3023
3024         if (isGauge[_gauge]) {
3025             uint256 _poolWeight = (_weights[i] * _weight) / _totalVoteWeight;
3026             if (votes[_tokenId][_pool] != 0) revert NonZeroVotes();
3027             if (_poolWeight == 0) revert ZeroBalance();
3028             //...
3029         }
3030     }
3031     //...
}

```

As a result, the pool can become permanently stuck in the **Active / Voting** state (when voting is done in batches), preventing completion of voting, epoch finalization, reward claiming and liquidity creation. All locked NFTs will remain trapped until an emergency withdrawal is executed, and the normal protocol flow is irreversibly broken.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:M/Y:M (6.3)

Recommendation

Enforce a non-zero minimum voting power at pool creation.

Remediation Comment

SOLVED: The **Kickoff.fun** team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.2 DEACTIVATED VEAERO NFTS CAN PERMANENTLY DOS VOTING BATCHES AND TRAP FUNDS

// MEDIUM

Description

`KickoffVoteSalePool.castVotesBatch()` calls `voter.vote()` for each locked `tokenId`. If Aerodrome deactivates a `tokenId`, the `voter.vote()` reverts for that `tokenId` and voting process will be permanently blocked. This can trap the pool in `Active / Voting` state (and possibly in a batch-in-progress state), preventing finalization and leaving project tokens permanently stuck, and accumulated rewards unclaimable under the intended flow (when previous batch calls succeeds), while users NFTs remain custody-held until an emergency recovery.

```
3044 // @ Voter.vote()
3045 // @ https://vscode.blockscan.com/8453/0x16613524e02ad97edfef371bc883f2f5d6c480a5
3046 function vote(
3047     uint256 _tokenId,
3048     address[] calldata _poolVote,
3049     uint256[] calldata _weights
3050 ) external onlyNewEpoch(_tokenId) nonReentrant {
3051     address _sender = _msgSender();
3052     if (!IVotingEscrow(ve).isApprovedOrOwner(_sender, _tokenId)) revert NotApprovedOrOwner();
3053     if (_poolVote.length != _weights.length) revert UnequalLengths();
3054     if (_poolVote.length > maxVotingNum) revert TooManyPools();
3055     if (IVotingEscrow(ve).deactivated(_tokenId)) revert InactiveManagedNFT();
3056     uint256 _timestamp = block.timestamp;
3057     if ((_timestamp > ProtocolTimeLibrary.epochVoteEnd(_timestamp)) && !isWhitelistedNFT[_tokenId])
3058         revert NotWhitelistedNFT();
3059     lastVoted[_tokenId] = _timestamp;
3060     uint256 _weight = IVotingEscrow(ve).balanceOfNFT(_tokenId);
3061     _vote(_tokenId, _weight, _poolVote, _weights);
3062 }
```

Proof of Concept

1. Update the `MockVotingEscrow.sol` to include a deactivated state and an accessor that mimics Aerodrome's deactivation behavior for veNFTs.

```
mapping(uint256 => bool) public deactivated;
function setManagedState(uint256 tokenId, bool _state) external {
    deactivated[tokenId] = _state;
}
```

2. Update `MockVoter.vote()` to enforce the same restriction as Aerodrome's Voter by reverting when voting is attempted with a deactivated veNFT.

```
1 import {MockVotingEscrow} from "../MockVotingEscrow.sol";
2 error InactiveManagedNFT();
3
4 function vote(uint256 tokenId, address[] calldata poolVote, uint256[] calldata _weights) external {
5     lastVoted[tokenId] = block.timestamp;
6     if (MockVotingEscrow(ve).deactivated(tokenId)) revert InactiveManagedNFT();
7
8     for (uint256 i = 0; i < poolVote.length; i++) {
9         votes[tokenId][poolVote[i]] = _weights[i];
10        weights[poolVote[i]] += _weights[i];
11    }
```



```

12 |         totalWeight += _weights[i];
13 |     }
    | }

```

3. Add `test_CastVotesRevertWithDeactivatedNFT()` test to `KickoffVoteSalePool.t.sol` that locks a deactivated veAERO NFT and attempts to progress the voting flow.

```

function test_CastVotesRevertWithDeactivatedNFT() public {
    // Setup
    vm.prank(admin);
    pool.activate();

    // the NFT is deactivated by the Aerodrome governance
    votingEscrow.setManagedState(1,true);

    vm.startPrank(user1);
    votingEscrow.approve(address(pool), 1);
    pool.lockVeAERO(1);
    vm.stopPrank();

    // Cast votes
    vm.prank(admin);
    vm.expectRevert(MockVoter.InactiveManagedNFT.selector);
    pool.castVotes(mockGauge);

    // the pool will stuck in Active state
    assertEq(uint256(pool.state()), uint256(KickoffVoteSalePool.PoolState.Active));

    // the admin can't finalize the process:
    vm.warp(8 days);
    vm.expectRevert(KickoffVoteSalePool.InvalidState.selector);
    pool.finalizeEpoch();
}

```

4. The test passes, demonstrating that locking a deactivated veAERO NFT is allowed and leads to a permanent denial of service: the pool remains stuck in `Active` state when voting is executed in a single batch, or becomes stuck in `Voting` when voting is executed across multiple batches and the deactivated NFT appears in a later batch (after `batchInProgress` is set to true).

```

$ forge test --mt test_CastVotesRevertWithDeactivatedNFT
Ran 1 test for test/KickoffVoteSalePool.t.sol:KickoffVoteSalePoolTest
[PASS] test_CastVotesRevertWithDeactivatedNFT() (gas: 341614)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.57ms (254.70µs CPU time)

Ran 1 test suite in 13.91ms (2.57ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)

```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:H/I:N/D:H/Y:H (5.6)

Recommendation

Harden batch voting against per-NFT failures by adding logic to skip NFTs that cannot vote and continue processing the rest, while recording skipped `tokenIds` for later handling or emergency return (where these NFTs are excluded from `totalVotingPower` and project token claiming).

Remediation Comment

SOLVED: The **Kickoff.fun** team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.3 EMERGENCY WITHDRAWALS BREAK CORE ACCOUNTING INVARIANTS AND CAN PERMANENTLY DEADLOCK THE POOL

// MEDIUM

Description

The emergency withdrawal functions `emergencyWithdrawNFT()` and `emergencyWithdrawBatch()` in `KickoffVoteSalePool` contract allow the owner to withdraw locked veAERO NFTs at any pool state without reconciling the pool's internal accounting or execution flow.

```
970 function emergencyWithdrawNFT(uint256 tokenId) external onlyOwner {
971     LockedNFT storage nft = lockedNFTs[tokenId];
972
973     if (nft.owner == address(0)) revert NFTNotLocked();
974     if (nft.unlocked) revert NFTNotLocked();
975
976     address nftOwner = nft.owner;
977     nft.unlocked = true;
978
979     // Transfer NFT back to original owner
980     votingEscrow.safeTransferFrom(address(this), nftOwner, tokenId);
981
982     emit EmergencyWithdraw(nftOwner, tokenId);
983 }
```

When an NFT is emergency-withdrawn, it is marked as unlocked and transferred back to its owner, but the `tokenId` remains in the pool's active NFT set (`lockedTokenIds`) and its voting power remains counted in both user-level and in `totalVotingPower` state. This breaks the invariant that all counted voting power corresponds to NFTs that actively participate in voting and reward generation.

During the voting phase, emergency withdrawals can disrupt batch voting execution by leaving a non-participating tokenIds in the voting set, causing the voting process to halt permanently. Once the pool is stuck in the `Voting` state, finalization cannot proceed, accumulated rewards cannot be claimed, liquidity is never created, and normal user exits are blocked.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:M (5.6)

Recommendation

Emergency withdrawal logic should be made state-aware and preserve all pool invariants by ensuring that withdrawn NFTs are fully excluded from voting, reward accounting (if withdrawn before voting), and batch execution in a manner consistent with the current pool state.

Remediation Comment

SOLVED: The `Kickoff.fun` team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d>

73

7.4 KICKOFF POOL CAN ACCEPT NON-VOTABLE NFTS DUE TO EPOCH-START MISMATCH BETWEEN KICKOFF AND AERODROME

// MEDIUM

Description

The `KickoffVoteSalePool` can be activated at any time, meaning the Kickoff epoch can start in the middle of an Aerodrome weekly epoch. In that situation, a user may have already voted earlier in the same Aerodrome epoch (before Kickoff activation), then later lock their veAERO NFT into Kickoff after activation.

```
294 function lockVeAERO(uint256 tokenId) external nonReentrant inState(PoolState.Active) {
295     // Check ownership
296     if (votingEscrow.ownerOf(tokenId) != msg.sender) {
297         revert NotNFTOwner();
298     }
299
300     // Check if NFT hasn't voted this epoch
301     uint256 lastVoted = voter.lastVoted(tokenId);
302     if (EpochLib.hasVotedThisEpoch(lastVoted)) {
303         revert AlreadyVotedThisEpoch();
304     }
305     //...
306 }
```

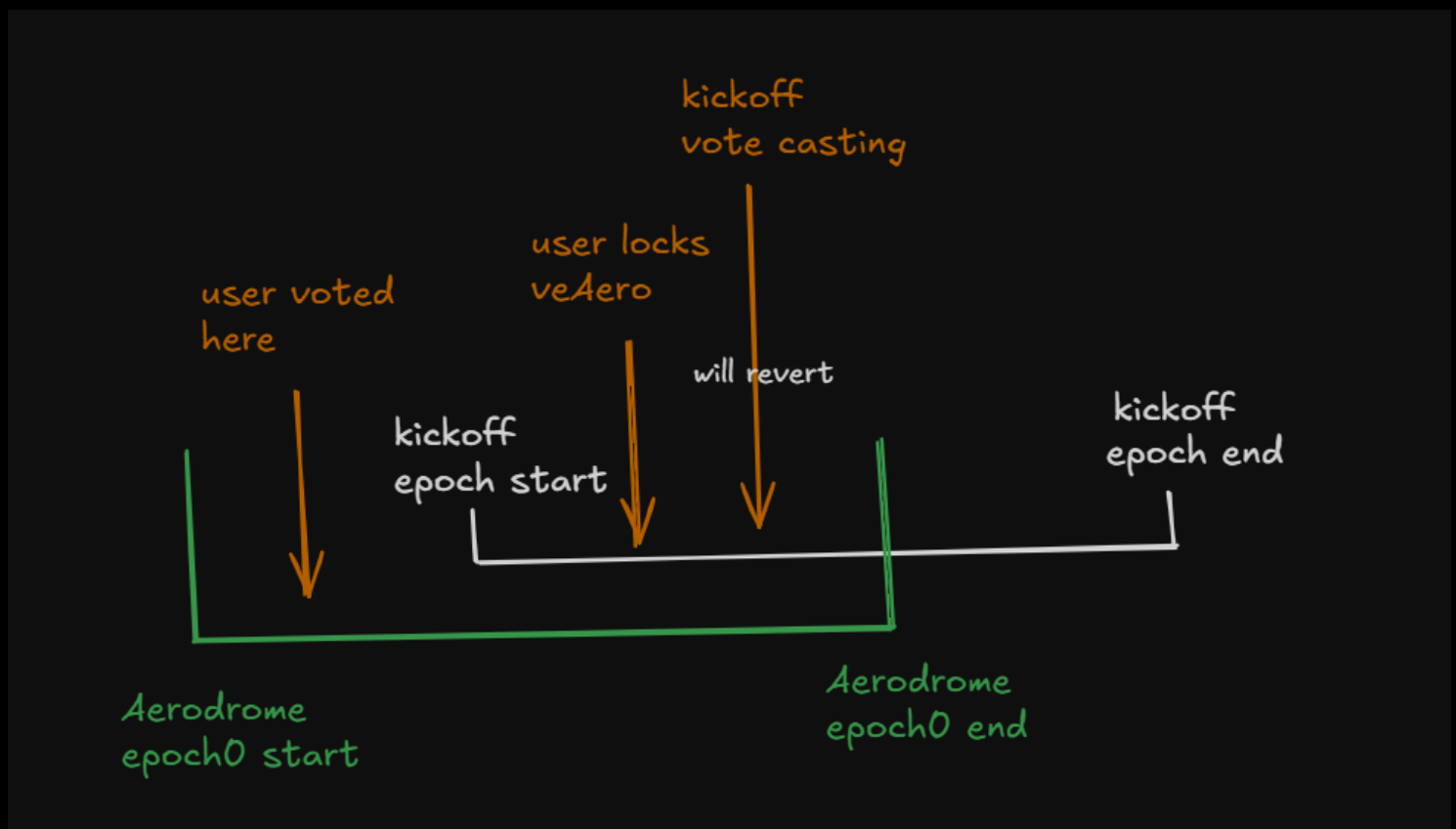
```
71 function hasVotedThisEpoch(
72     uint256 lastVotedTimestamp
73 ) internal view returns (bool) {
74     // Using > (not >=) to match Aerodrome's exact logic
75     // NFT can vote again if lastVoted == epochStart (edge case)
76     return lastVotedTimestamp > currentEpochStart();
77 }
```

```
2893 // @voter.onlyNewEpoch
2894 // https://vscode.blockscan.com/8453/0x16613524e02ad97edfef371bc883f2f5d6c480a5
2895 modifier onlyNewEpoch(uint256 _tokenId) {
2896     // ensure new epoch since last vote
2897     if (ProtocolTimeLibrary.epochStart(block.timestamp) <= lastVoted[_tokenId]) revert Already
2898     if (block.timestamp <= ProtocolTimeLibrary.epochVoteStart(block.timestamp)) revert Distrib
2899     -;
2900 }
2901
2902 function epochVoteStart(uint256 timestamp) internal pure returns (uint256) {
2903     unchecked {
2904         return timestamp - (timestamp % WEEK) + 1 hours;
2905     }
2906 }
```

```
3044 // @voter.onlyNewEpoch
3045 // https://vscode.blockscan.com/8453/0x16613524e02ad97edfef371bc883f2f5d6c480a5
3046 function vote(
3047     uint256 _tokenId,
3048     address[] calldata _poolVote,
3049     uint256[] calldata _weights
3050 ) external onlyNewEpoch(_tokenId) nonReentrant {...}
```

Example:

- Aerodrome epoch0 starts.
- The user votes with their NFT during Aerodrome epoch0 (so `lastVoted[tokenId]` is set within epoch0).
- Kickoff activates later (Kickoff epoch start is after Aerodrome epoch0 start).
- The user locks the NFT after Kickoff activation.
- Under Kickoff's epoch start notion, the NFT looks like it hasn't voted this Kickoff epoch, so the lock is allowed.
- The admin then calls Kickoff vote casting (still within Aerodrome epoch0).
- Aerodrome's voter enforces onlyNewEpoch, which requires a new Aerodrome epoch since last vote (`epochStart(now) > lastVoted[tokenId]`).
- Because the user already voted earlier in Aerodrome epoch0, this condition fails and `voter.vote()` reverts with `AlreadyVotedOrDeposited()` error.



BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:M (5.6)

Recommendation

Require pool activation to be aligned with Aerodrome's epoch start by explicitly storing the Aerodrome `epochStart` at activation, and enforce that both NFT locking eligibility and vote execution are constrained to that same Aerodrome epoch.

Remediation Comment

SOLVED: The **Kickoff.fun** team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.5 POOL CREATION CAN BE FRONT-RUN TO STEAL PRIVILEGED ROLES AND BLOCK THE INTENDED LAUNCH

// MEDIUM

Description

`KickoffFactory.createPool()` is permissionless and allows any caller to create a pool for a given `projectToken`. Since only one pool per `projectToken` is allowed, an attacker can front-run the intended deployment, become the pool admin and project owner, and permanently prevent the real project team from creating their own pool. This also lets the attacker capture the generated fees for that pool.

```
108 function createPool(  
109     address projectToken,  
110     address projectOwner,  
111     uint256 totalAllocation,  
112     uint256 minVotingPower  
113 ) external returns (address pool) {  
114     //...  
115  
116     // Deploy new pool  
117     // msg.sender becomes the admin (receives 30% of trading fees)  
118     pool = address(  
119         new KickoffVoteSalePool(  
120             projectToken,  
121             msg.sender, // admin  
122             projectOwner,  
123             totalAllocation,  
124             minVotingPower,  
125             address(lpLocker),  
126             votingEscrow,  
127             voter,  
128             router,  
129             weth  
130         )  
131     );  
132     //...  
133 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

Restrict `createPool()` to the factory owner.

Remediation Comment

SOLVED: The `Kickoff.fun` team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

7.6 LIQUIDITY FINALIZATION CAN BE DOSED DUE TO MANIPULATED POOL RATIOS

// MEDIUM

Description

During finalization, the kickoff pool adds liquidity via `router.addLiquidity()` using fixed amounts of project tokens and collected WETH, together with strict minimum amounts derived from a slippage tolerance. If the Aerodrome pool already exists, the router computes optimal deposit amounts based on the pool's current reserves and enforces the provided minimums.

Because pool creation and initial liquidity provisioning on Aerodrome are permissionless, a third party can pre-create and pre-seed the pool with an extreme or unfavorable token ratio. In such a case, the optimal amounts computed by the router may fall below the minimum thresholds, causing the liquidity addition to revert.

```
866 function _addLiquidity() internal {
867     //...
868
869     // Calculate minimum amounts with slippage tolerance
870     uint256 minProjectToken = (liquidityAllocation * (BPS_DENOMINATOR - liquiditySlippageBps))
871     uint256 minWeth = (wethCollected * (BPS_DENOMINATOR - liquiditySlippageBps)) / BPS_DENOMIN
872
873     // Add liquidity (volatile pool) with slippage protection
874     (, uint256 liquidity) = router.addLiquidity(
875         projectToken,
876         wethAddr,
877         false, // volatile
878         liquidityAllocation,
879         wethCollected,
880         minProjectToken,
881         minWeth,
882         address(this),
883         block.timestamp
884     );
885     //...
886 }
```

```
2311 // @router._addLiquidity()
2312 // @https://vscode.blockscan.com/8453/0xcF77a3Ba9A5CA399B7c97c74d54e5b1Beb874E43
2313 function _addLiquidity(
2314     address tokenA,
2315     address tokenB,
2316     bool stable,
2317     uint256 amountADesired,
2318     uint256 amountBDesired,
2319     uint256 amountAMin,
2320     uint256 amountBMin
2321 ) internal returns (uint256 amountA, uint256 amountB) {
2322     if (amountADesired < amountAMin) revert InsufficientAmountADesired();
2323     if (amountBDesired < amountBMin) revert InsufficientAmountBDesired();
2324     // create the pool if it doesn't exist yet
2325     address _pool = IPoolFactory(defaultFactory).getPool(tokenA, tokenB, stable);
2326     if (_pool == address(0)) {
2327         _pool = IPoolFactory(defaultFactory).createPool(tokenA, tokenB, stable);
2328     }
2329     (uint256 reserveA, uint256 reserveB) = getReserves(tokenA, tokenB, stable, defaultFactory);
2330     if (reserveA == 0 && reserveB == 0) {
2331         (amountA, amountB) = (amountADesired, amountBDesired);
2332     } else {
2333         (amountA, amountB) = IPoolFactory(defaultFactory).getOptimalLiquidity(
2334             tokenA, tokenB, stable, amountADesired, amountBDesired, amountAMin, amountBMin);
2335     }
2336 }
```

```

2334     uint256 amountBOptimal = quoteLiquidity(amountADesired, reserveA, reserveB);
2335     if (amountBOptimal <= amountBDesired) {
2336         if (amountBOptimal < amountBMin) revert InsufficientAmountB();
2337         (amountA, amountB) = (amountADesired, amountBOptimal);
2338     } else {
2339         uint256 amountAOptimal = quoteLiquidity(amountBDesired, reserveB, reserveA);
2340         assert(amountAOptimal <= amountADesired);
2341         if (amountAOptimal < amountAMin) revert InsufficientAmountA();
2342         (amountA, amountB) = (amountAOptimal, amountBDesired);
2343     }
2344 }

```

This blocks LP creation, prevents epoch finalization, and leaves the pool stuck with rewards already claimed (when finalization is done over multiple calls) but liquidity never created.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

Liquidity finalization should be designed to safely handle pre-existing pool state, ensuring that hostile or unexpected reserve ratios cannot permanently block progress or completion of the pool lifecycle.

Remediation Comment

SOLVED: The **Kickoff.fun** team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.7 POOLS CAN FINALIZE WITH ZERO PARTICIPANTS AND PERMANENTLY LOCK PROJECT TOKENS

// MEDIUM

Description

If no veAERO NFTs are locked into the kickoff pool, the protocol still allows the pool to progress through activation, voting, and finalization successfully. In this scenario, no votes are cast, no rewards are earned, no liquidity is created, and no participants to claim project tokens.

After completion, the `KickoffVoteSalePool` contract still holds the full project token allocation, but rescuing the project token is explicitly disallowed. This leaves the project token balance permanently locked in a completed pool with zero participation.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:M/Y:N (5.0)

Recommendation

The pool should implement an explicit handling path for the zero-participant completion scenario that avoids permanently locking project tokens and enables safe recovery when no distribution or liquidity provisioning occurred.

Remediation Comment

SOLVED: The `Kickoff.fun` team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.8 UNINTENDED LIQUIDITY CREATION CAN PERMANENTLY CONSUME PROJECT TOKENS DUE TO EXTERNAL WETH TRANSFERS

// MEDIUM

Description

The pool's finalization logic implicitly trusts the contract's WETH balance when adding liquidity. If no veAERO NFTs are locked, the pool can still proceed to finalization, and normally no liquidity would be created because no rewards are earned. However, if a third party donates even a small amount of WETH to the `KickoffVoteSalePool` contract before finalization, the pool will treat this balance as valid input for liquidity creation.

During finalization, `_addLiquidity()` will pair the full project token liquidity allocation (`liquidityAllocation`) with the donated WETH, potentially creating highly imbalanced and economically meaningless liquidity.

This liquidity is then permanently locked in the LP locker, irreversibly consuming project tokens despite zero participation, zero votes, and zero earned rewards. The remaining project tokens (`saleAllocation`) will be permanently locked, with no recovery path. This allows an external actor to grief the system and cause permanent loss of project funds at negligible cost.

```
866 function _addLiquidity() internal {
867     if (wethCollected == 0) return;
868     //...
869
870     // Add liquidity (volatile pool) with slippage protection
871     (, , uint256 liquidity) = router.addLiquidity(
872         projectToken, //! tokenA
873         wethAddr, //! tokenB
874         false, // volatile
875         liquidityAllocation,
876         wethCollected,
877         minProjectToken,
878         minWeth,
879         address(this),
880         block.timestamp
881     );
882
883     //...
884 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:M/Y:N (5.0)

Recommendation

Liquidity creation during finalization should be conditioned on meaningful participation and reward generation, and should not rely solely on externally transferable WETH balance.

Remediation Comment

SOLVED: The **Kickoff.fun** team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.9 REWARD CONVERSION MAY EXECUTE WITH UNBOUNDED SLIPPAGE

// MEDIUM

Description

During reward conversion, the contract computes `amountOutMin` using `_getMinOutputWithSlippage()`. If `router.getAmountsOut()` fails or returns a value that rounds to zero after applying slippage, the function returns `1 wei` as the minimum output. This causes swaps to proceed with no slippage protection, allowing execution at unfavorable prices as long as the swap returns at least 1 wei.

```
854 function _getMinOutputWithSlippage(  
855     uint256 amountIn,  
856     IRouter.Route[] memory routes  
857 ) internal view returns (uint256) {  
858     try router.getAmountsOut(amountIn, routes) returns (  
859         uint256[] memory amounts  
860     ) {  
861         if (amounts.length > 1 && amounts[amounts.length - 1] > 0) {  
862             // Apply slippage tolerance  
863             uint256 minOut = (amounts[amounts.length - 1] *  
864                 (BPS_DENOMINATOR - swapSlippageBps)) / BPS_DENOMINATOR;  
865             return minOut > 0 ? minOut : 1; // Ensure at least 1 wei  
866         }  
867     } catch {}  
868     return 1; // Minimum 1 wei to protect against dust/sandwich attacks  
869 }
```

This behavior is particularly risky when a pool (route) exists but is illiquid or temporarily manipulated, as the swap can succeed and silently realize significant value loss. Importantly, the no pool/invalid route case is already handled safely by the try/catch logic during swapping, where the swap reverts (caught in the catch block) and the reward tokens remain in the contract to be rescued later. As a result, falling back to `minOut = 1` is not required for liveness and instead does not provide a meaningful slippage protection.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

When a reliable quote cannot be obtained, avoid executing the swap rather than degrading slippage protection. Prefer skipping the swap for that token and rely on the existing rescue path, ensuring reward conversion only occurs when meaningful slippage bounds can be enforced.

Remediation Comment

SOLVED: The `Kickoff.fun` team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

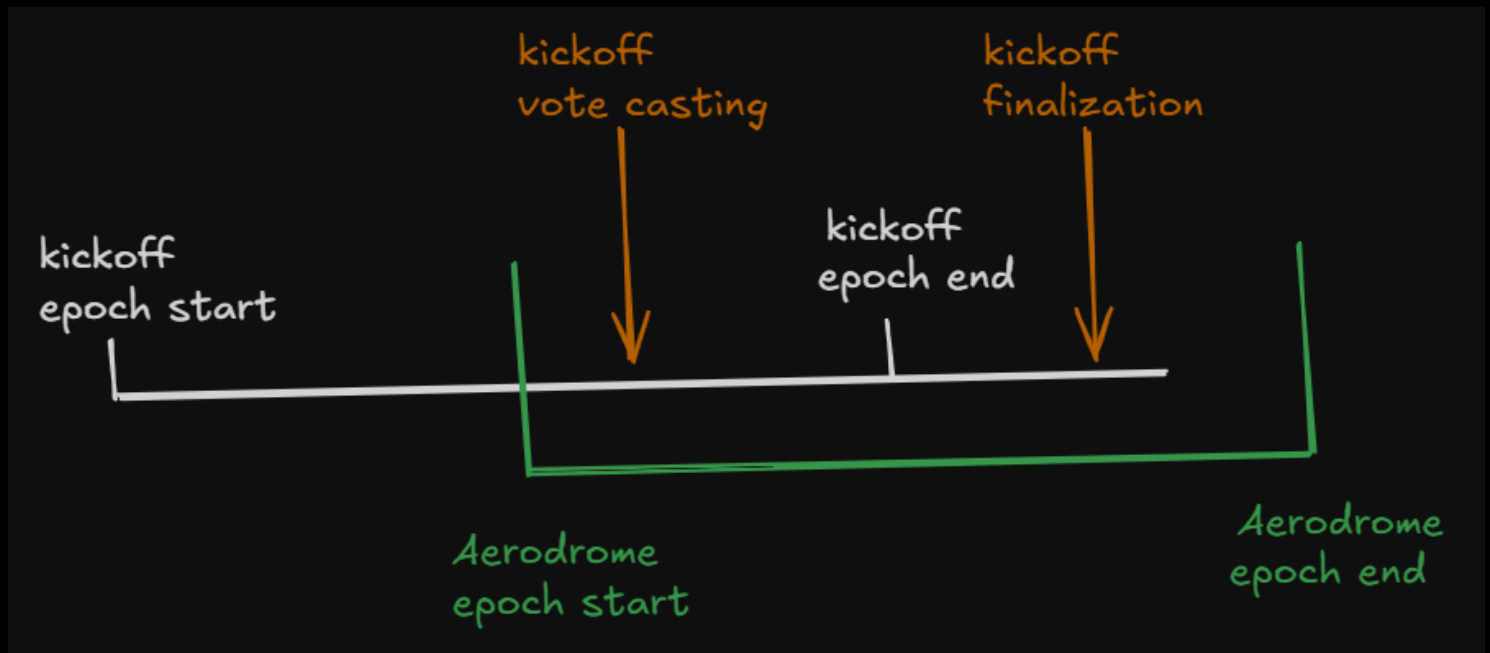
<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.10 KICKOFF FINALIZATION CAN OCCUR BEFORE AERODROME EPOCH COMPLETION, CAUSING REWARD LOSS

// MEDIUM

Description

The `KickoffVoteSalePool` measures epoch completion relative to its own activation time rather than Aerodrome's epoch boundaries. While both systems use one-week epochs, Kickoff pools can be activated at any point within an ongoing Aerodrome epoch.



If Kickoff is activated before Aerodrome's epoch boundary, Kickoff's internal epoch end (derived from its `activeEpoch`) can occur earlier than the end of the Aerodrome epoch in which votes are later cast. In that case, the admin can reach Kickoff's finalize condition and call `finalizeEpoch()` / `startClaimRewardsBatch()` while the Aerodrome epoch that received the votes is still in progress.

Because Aerodrome reward accounting for vote-based rewards only becomes claimable across completed epochs, claiming during Kickoff finalization returns zero even though voting successfully occurred, leading to missing rewards and no liquidity provisioning.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

Voting and finalization should be explicitly constrained to ensure alignment with Aerodrome epoch boundaries, so that finalization cannot occur until the Aerodrome epoch in which votes were cast has fully completed and rewards are guaranteed to be accrued.

Remediation Comment

SOLVED: The **Kickoff.fun** team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.11 GAUGE STATUS CHANGE BETWEEN VOTE BATCHES CAN PERMANENTLY BRICK VOTING AND TRAP NFTS

// LOW

Description

`KickoffVoteSalePool.castVotesBatch()` validates that the `gauge` is alive only on the first batch. If voting requires multiple batch calls and the gauge becomes killed between calls, subsequent `voter.vote()` calls will revert, permanently preventing batch completion. The pool can remain stuck in `Voting` state with NFTs custody-held, preventing normal progress to finalization and liquidity creation.

```
340 function castVotesBatch(address _gauge, uint256 batchSize) external onlyAdmin {
341     // First batch requires Active state, subsequent batches require Voting state with batch i
342     if (!batchInProgress) {
343         if (state != PoolState.Active) revert InvalidState();
344     } else {
345         if (state != PoolState.Voting) revert InvalidState();
346     }
347
348     if (batchSize > MAX_BATCH_SIZE) revert BatchSizeTooLarge();
349
350     uint256 length = lockedTokenIds.length;
351     if (length == 0) {
352         _setState(PoolState.Voting);
353         emit VotesCast(_gauge, 0);
354         return;
355     }
356
357     // First batch - initialize and change state immediately
358     if (!batchInProgress) {
359         if (_gauge == address(0)) revert ZeroAddress();
360
361         // Validate gauge
362         address pool = voter.poolForGauge(_gauge);
363         if (pool == address(0)) revert InvalidGauge();
364         if (!voter.isAlive(_gauge)) revert GaugeNotActive();
365
366         gauge = _gauge;
367         aerodromePool = pool;
368
369         // Note: bribe addresses are fetched dynamically during claim via gaugeToFees/gaugeToB
370
371         batchIndex = 0;
372         batchInProgress = true;
373
374         // Change state to Voting immediately - this blocks further NFT locks
375         _setState(PoolState.Voting);
376     }
377     //...
378 }
```

```
304 // @ Voter._vote()
305 // @ https://vscode.blockscan.com/8453/0x16613524e02ad97edfef371bc883f2f5d6c480a5
306 function _vote(uint256 _tokenId, uint256 _weight, address[] memory _poolVote, uint256[] memory
307     _reset(_tokenId);
308     uint256 _poolCnt = _poolVote.length;
309     uint256 _totalVoteWeight = 0;
310     uint256 _totalWeight = 0;
311     uint256 _usedWeight = 0;
312
313     for (uint256 i = 0; i < _poolCnt; i++) {
314         _totalVoteWeight += _weights[i];
315     }
316 }
```

```

317     for (uint256 i = 0; i < _poolCnt; i++) {
318         address _pool = _poolVote[i];
319         address _gauge = gauges[_pool];
320         if (_gauge == address(0)) revert GaugeDoesNotExist(_pool);
321         if (!isAlive[_gauge]) revert GaugeNotAlive(_gauge);
322
323         //...
324     }
325     //...
326 }

```

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:N/I:N/D:M/Y:M (3.1)

Recommendation

Revalidate gauge liveness on every batch call, and add a safe skip mechanism that allows completing the voting phase (or reverting to a recoverable state) if a gauge becomes invalid (killed) mid-process.

Remediation Comment

SOLVED: The **Kickoff.fun** team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.12 CLAIM ROUNDING LEAVES IRRECOVERABLE PROJECT TOKEN DUST IN THE POOL

// LOW

Description

`KickoffVoteSalePool.claimProjectTokens()` uses integer division to compute each user's share. This rounding down can leave leftover dust of `projectToken` in the pool after all users claim. Because `rescueTokens()` explicitly prevents rescuing `projectToken`, this dust becomes permanently stuck.

```
932 function claimProjectTokens() external nonReentrant inState(PoolState.Completed) {
933     UserInfo storage user = userInfo[msg.sender];
934
935     if (user.claimed) revert AlreadyClaimed();
936     if (user.totalVotingPower == 0) revert NothingToClaim();
937
938     user.claimed = true;
939
940     // Calculate user's share of sale allocation
941     uint256 userShare = (saleAllocation * user.totalVotingPower) / totalVotingPower;
942
943     // Transfer project tokens
944     if (!IERC20(projectToken).transfer(msg.sender, userShare)) {
945         revert TransferFailed();
946     }
947
948     emit ProjectTokensClaimed(msg.sender, userShare);
949 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (2.5)

Recommendation

Ensure the full `saleAllocation` can be distributed by tracking claimed totals and assigning the remainder deterministically (last claimer gets remainder).

Remediation Comment

SOLVED: The `Kickoff.fun` team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d>

7.13 SWAP AND LIQUIDITY OPERATIONS LACK MEANINGFUL DEADLINE CONSTRAINTS

// LOW

Description

During finalization, swaps and liquidity addition pass `block.timestamp` as the deadline. This does not provide meaningful protection against stale or delayed execution because the deadline is always **now** at the time of execution. This reduces control over when finalization actions are valid and can result in an undesirable swaps/liquidity outcomes (as minimums are calculated based on the current quotes).

```
802 function _convertToWETHInternal(address[] memory rewardTokens) internal {
803     //...
804     for (uint256 i = 0; i < length;) {
805         //...
806         try router.swapExactTokensForTokens(balance, minOut, routes, address(this), block.
807             catch {
808                 // Try stable route with slippage
809                 routes[0].stable = true;
810                 minOut = _getMinOutputWithSlippage(balance, routes);
811                 try router.swapExactTokensForTokens(balance, minOut, routes, address(this), bl
812                     catch {
813                         // Token stays on contract for manual rescue
814                     }
815                 }
816             }
817         unchecked { ++i; }
818     }
819 }
820 //...
821 }
822 }
```

```
866 function _addLiquidity() internal {
867     //...
868     // Add liquidity (volatile pool) with slippage protection
869     (, uint256 liquidity) = router.addLiquidity(
870         projectToken,
871         wethAddr,
872         false, // volatile
873         liquidityAllocation,
874         wethCollected,
875         minProjectToken,
876         minWeth,
877         address(this),
878         block.timestamp
879     );
880     //...
881 }
882 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

Use a deadline chosen by the admin (or derived from a configured window) and passed explicitly into finalization functions.

Remediation Comment

SOLVED: The **Kickoff.fun** team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.14 RESCUE MECHANISM FAILS FOR TOKENS THAT DO NOT RETURN A BOOLEAN ON SUCCESSFUL TRANSFERS

// LOW

Description

`KickoffVoteSalePool.rescueTokens()` relies on `IERC20(token).transfer()` returning a boolean. Some ERC20 tokens do not return a boolean value on transfers, which causes the call to revert, preventing rescue of stuck tokens. This is especially relevant because reward tokens obtained from Aerodrome incentives can be diverse and not guaranteed to follow strict ERC20 return conventions.

```
1052 function rescueTokens(address token, address to, uint256 amount) external onlyAdmin {
1053     if (token == projectToken) revert NotProjectToken();
1054     if (to == address(0)) revert ZeroAddress();
1055
1056     if (!IERC20(token).transfer(to, amount)) {
1057         revert TransferFailed();
1058     }
1059
1060     emit TokensRescued(token, to, amount);
1061 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

Use a safe transfer wrapper (`safeTransfer()` from the OpenZeppelin `SafeERC20` library) to ensure rescue succeeds across common token implementations.

Remediation Comment

SOLVED: The `Kickoff.fun` team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d>

7.15 BATCH VOTING CAN SPAN MULTIPLE AERODROME EPOCHS, LEADING TO INCONSISTENT REWARD ATTRIBUTION

// LOW

Description

`KickoffVoteSalePool.castVotesBatch()` allows voting to be executed across multiple transactions without enforcing that all votes occur within the same Aerodrome epoch. If batching begins near an epoch boundary, the Aerodrome epoch may roll over between batch calls, causing different locked NFTs to cast votes in different epochs.

Aerodrome rewards are calculated strictly per epoch and only accrue after an epoch completes. As a result, NFTs voted earlier may accrue rewards for one epoch while NFTs voted later accrue rewards for a subsequent epoch. However, Kickoff finalization assumes a single, coherent voting epoch tied to `activeEpoch` and does not account for votes spanning multiple Aerodrome epochs.

When finalization occurs, reward claiming return zero rewards for NFTs whose votes belong to a later epoch that has not yet ended, resulting in missed rewards and reduced liquidity provisioning despite valid voting activity.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

Voting execution should be explicitly constrained to a single, well-defined Aerodrome epoch to ensure consistency between pool state, voting behavior, and reward attribution.

Remediation Comment

SOLVED: The `Kickoff.fun` team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.16 LOCKING VEAERO NFTS ALLOWS KICKOFF TO CLAIM HISTORICAL VOTING REWARDS

// LOW

Description

When a user locks a veAERO NFT into `KickoffVoteSalePool`, ownership of the NFT is transferred to the Kickoff contract. Aerodrome voting rewards (fees and bribes) are distributed per `tokenId` and are always paid to the `ownerOf(tokenId)` at claim time. During finalization, Kickoff calls `voter.claimBribes()`, which is authorized because Kickoff is the current NFT owner (the ability to claim historical rewards follows the ownership of the veAERO).

As a result, any unclaimed voting rewards accrued before the NFT was locked can be claimed by Kickoff and redirected into the pool's finalization flow. This silently deprives the original NFT owner of rewards they earned independently of the Kickoff participation.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:L (2.5)

Recommendation

Prevent veAERO NFTs from being locked into `KickoffVoteSalePool` while they have unclaimed voting rewards.

Remediation Comment

RISK ACCEPTED: The `Kickoff.fun` team accepted the risk of this finding.

7.17 LPLOCKER FEE DISTRIBUTION CAN BE PERMANENTLY DOSED BY RESTRICTIVE PROJECT TOKEN BEHAVIOR

// LOW

Description

`LPLocker.claimTradingFees()` distributes project's pool trading fees by transferring the pool's two underlying tokens (`projectToken` and WETH) to fixed recipients (`admin` and `projectOwner`). If the project token enforces a blacklist restriction and either the admin or project owner becomes blacklisted, the transfer of that token will revert. Because both transfers are executed in the same flow and recipients are immutable, this will block the entire function.

As a result, fee claiming becomes permanently unavailable, preventing both parties from receiving accrued fees.

```
139 function claimTradingFees(address votePool) external nonReentrant {
140     //...
141     // Transfer to admin
142     if (adminShare0 > 0) {
143         if (!IERC20(token0).transfer(pool.admin, adminShare0))
144             revert TransferFailed();
145     }
146     if (adminShare1 > 0) {
147         if (!IERC20(token1).transfer(pool.admin, adminShare1))
148             revert TransferFailed();
149     }
150
151     // Transfer to project owner
152     if (projectShare0 > 0) {
153         if (!IERC20(token0).transfer(pool.projectOwner, projectShare0))
154             revert TransferFailed();
155     }
156     if (projectShare1 > 0) {
157         if (!IERC20(token1).transfer(pool.projectOwner, projectShare1))
158             revert TransferFailed();
159     }
160     //...
161 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:L (2.5)

Recommendation

Consider implementing a pull-based mechanism: claim and account for accrued trading fees internally for the admin and project owner, and provide separate function that allow each party to withdraw their accumulated balances to a receiver address of their choosing.

Remediation Comment

SOLVED: The `Kickoff.fun` team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d>

73

7.18 LEFTOVER LIQUIDITY ALLOCATION TOKENS CAN BECOME PERMANENTLY STUCK AFTER ADDING LIQUIDITY

// LOW

Description

`KickoffVoteSalePool._addLiquidity()` supplies `liquidityAllocation` of `projectToken` and `wethCollected` to `router.addLiquidity()`, but the router may use less than the desired amounts to match the pool's current reserves (it returns (amountA, amountB, liquidity) internally and only transfers the optimal amounts). Any unused remainder stays in the `KickoffVoteSalePool` contract.

Since `rescueTokens()` explicitly prevents rescuing `projectToken`, any leftover project tokens after liquidity provisioning become permanently stuck.

```
866 function _addLiquidity() internal {
867     //...
868     // Approve tokens
869     IERC20(projectToken).approve(routerAddr, liquidityAllocation);
870     weth.approve(routerAddr, wethCollected);
871
872     // Calculate minimum amounts with slippage tolerance
873     uint256 minProjectToken = (liquidityAllocation *
874         (BPS_DENOMINATOR - liquiditySlippageBps)) / BPS_DENOMINATOR;
875     uint256 minWeth = (wethCollected *
876         (BPS_DENOMINATOR - liquiditySlippageBps)) / BPS_DENOMINATOR;
877
878     // Add liquidity (volatile pool) with slippage protection
879     (, , uint256 liquidity) = router.addLiquidity(
880         projectToken,
881         wethAddr,
882         false, // volatile
883         liquidityAllocation,
884         wethCollected,
885         minProjectToken,
886         minWeth,
887         address(this),
888         block.timestamp
889     );
890     //...
891 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (2.5)

Recommendation

Consider adding any unspent `liquidityAllocation` after successful liquidity provisioning to the `saleAllocation` (and ensure there are participants that will claim these sale allocation), so these leftovers cannot remain stuck in the pool contract.

Remediation Comment

SOLVED: The **Kickoff.fun** team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d73>

7.19 EARLY EMERGENCY EXIT CAN PERMANENTLY LOCK ALLOCATED PROJECT TOKENS AND ALLOW NEW LOCKS INTO AN UNUSABLE POOL

// LOW

Description

If the `KickoffVoteSalePool` owner triggers an emergency `emergencyWithdrawAllNFTs()` before voting starts (while the pool is in `Active` state), NFTs can be returned, but the pool still holds the sale and liquidity allocations of `projectToken` intended for distribution and LP creation.

Because the contract disallows rescuing `projectToken`, and there is no explicit shutdown/cancel pool flow, an emergency exit intended to abandon the pool can leave allocated `projectToken` stuck in the pool indefinitely, and the pool still able to accept new locks later, creating a scenario where users could lock NFTs into a pool that will never progress to a claimable/finalized state.

```
986 function emergencyWithdrawAllNFTs() external onlyOwner {
987     if (batchInProgress) revert BatchInProgress();
988
989     uint256 length = lockedTokenIds.length;
990     for (uint256 i = 0; i < length; ) {
991         _emergencyWithdrawSingle(lockedTokenIds[i]);
992         unchecked {
993             ++i;
994         }
995     }
996 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N (2.5)

Recommendation

Add an explicit cancel/shutdown path that permanently disables further locking of veAero NFTs (irreversible state) and enables recovery of `projectToken` to a designated recipient (project owner) when the pool is cancelled while in the `Active` state.

Remediation Comment

SOLVED: The `Kickoff.fun` team solved the issue in the specified commit id by following the mentioned recommendation.

Remediation Hash

<https://github.com/kickoffbase/kickoff-contracts/commit/98e47ac039330f31cdf2f1dfb2ef17b4a2f97d>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.